Git & GitHub — Interview Revision Notes

Concepts, commands and common interview questions on one sheet

1. What is Git?

- **Git** is a **distributed version control system (VCS)**. It stores snapshots of your codebase over time.
- **Why:** if a new release has bugs, you can go back to the last stable version.
- **VCS types:** *Centralized* (one central server, e.g. SVN) vs *Distributed* (every developer has the full history, e.g. Git).
- **Advantages:** version history, easy bug fixing / rollback, non-linear development (different features by different people in parallel), collaboration.
- **Git vs GitHub:** Git is the tool on your PC. GitHub (also GitLab, Bitbucket) hosts remote repos and adds PRs, issues, actions.

2. The Three Areas

```
Working Directory  --git add-->  Staging Area  --git commit-->  Repository
(edit files)                       (next commit)                (saved history)
```

- File states in `git status`: **Untracked** (new, Git doesn't know it), **Modified** (changed since last commit), **Staged/Added** (ready to commit).
- After a commit, the file is clean and VS Code removes the U/M/A marker.
- **When to commit:** after a meaningful unit of work (a feature or a fix), not after every line.
- **Commit messages:** short, imperative. Write it as *"This commit will ___"* without those words, e.g. `Add login validation`.

3. Basic Commands

Command	What it does
`git init`	Start tracking the current folder as a repo (creates .git)
`git clone <url>`	Copy a remote repo (full history) to your PC
`git status`	Show untracked, modified and staged files
`git add <file>`	Move a file to the staging area
`git add .`	Stage all untracked + modified files at once
`git commit -m "msg"`	Save staged changes into the repo as a commit
`.gitignore`	List files Git should not track (e.g. .env, venv/). The .gitignore file itself is committed
`rm -rf .git`	Remove Git from a folder (undo a wrong git init)

4. Viewing History

Command	What it does
`git log`	All commits: SHA (40-char unique ID), author, date, message. Press q to exit
`git log --oneline`	One line per commit, short SHA
`git log --stat`	Per file: how many lines added / deleted
`git log -p`	Full code diff for every commit
`git show <sha>`	Code changes of one commit (short or full SHA)
`git diff`	Working directory vs last commit (unstaged changes)
`git diff --staged`	Staged changes vs last commit
`git log --all --oneline --graph`	Graph view of all branches
`git log --author="Name"`	Commits by one person
`git log --grep="Regression"`	Commits whose message contains a word
`git shortlog -n -s`	Contributors with their commit counts, sorted

5. Tags & Versioning

- **Semantic versioning X.Y.Z**: X = Major (breaking, not backward compatible, e.g. 2.x vs 3.x), Y = Minor (new features, compatible), Z = Patch (small bug fixes, e.g. 1.5.0 → 1.5.1).
- A **tag** is a permanent label on a commit, usually a release version.

Command	What it does
`git tag -a v1.0.0 -m "msg"`	Annotated tag on the latest commit (without -m, an editor opens)
`git tag -a v1.0.0 <sha>`	Tag an older commit
`git tag`	List tags
`git tag -d v1.0.0`	Delete a tag
`git push origin --tags`	Push tags to remote (not pushed by default)

6. Branching

- A **branch** lets work happen in parallel without touching main. Default branch was *master*, now usually *main*.
- **HEAD** = pointer to the latest commit of the branch you're currently on.
- Switching branches changes the files in your folder/IDE to that branch's version.
- A branch can start from an old commit, e.g. latest commit broke something: branch from the previous good commit, fix, merge back.

Command	What it does
`git branch`	List branches; * marks the current one
`git branch -a`	Include remote branches
`git branch <name>`	Create a branch at the current commit
`git branch <name> <sha>`	Create a branch from an older commit
`git checkout <name>`	Switch branch (newer: git switch)
`git checkout -b <name>`	Create + switch (newer: git switch -c)
`git branch -d <name>`	Delete; warns/refuses if not merged
`git branch -D <name>`	Force delete, no warning

7. Merging

- Merge happens **into the branch you are on**: `git checkout main` then `git merge feature`.
- **Fast-forward merge:** main didn't change since branching, so Git just moves the main pointer ahead. No new commit.
- **Regular (3-way / divergent) merge:** both branches have new commits. Git creates a new **merge commit** combining them.
- Different lines changed on each side → merges automatically. Same lines changed → **merge conflict**.

Conflict markers in the file:

```
<<<<<<< HEAD
print("Hi there")        <- your current branch
=======
print("Hello, Yash")     <- incoming branch
>>>>>>> feature
```

Resolve: edit the file to the final version, delete all marker lines, then `git add <file>` and `git commit`. To cancel: `git merge --abort`. Error *"not something we can merge"* = branch name is wrong or doesn't exist; check `git branch -a`.

8. Working with a Remote (GitHub)

- Workflow: **pull** latest code → make changes → commit → **push**.
- **origin** is just the default nickname for the remote URL.
- `origin/main` in `git log` shows how far the remote has your commits; `main` shows your local position.

Command	What it does
`git remote add origin <url>`	Connect a local repo to a GitHub repo

Command	What it does
`git remote -v`	Show remote URLs (fetch/push)
`git push -u origin main`	Upload commits; -u sets upstream so later just git push
`git fetch`	Download remote changes, don't touch your files
`git pull origin main`	fetch + merge into your current branch

9. Undoing Changes

Pick the command by **where the change is**: working dir → staged → committed (local) → pushed. **Rule:** `reset` rewrites history, so use it only on local commits; `revert` adds a new undo commit, so it is safe after push.

Command	What it does
`git restore <file>`	Discard unstaged edits in a file. **Use when:** you messed up a file and want the last committed version back. Edits are lost for good
`git restore --staged <file>`	Unstage a file, edits stay. **Use when:** you ran git add on the wrong file
`git commit --amend`	Replace the last commit (new SHA). **Use when:** wrong message or forgot a file, and not pushed yet
`git reset --soft HEAD~1`	Remove last commit; changes stay **staged**. **Use when:** you want to redo the commit, or squash several small commits into one
`git reset HEAD~1`	(--mixed, the default) Remove last commit; changes stay in working dir but **unstaged**. **Use when:** you committed too early or committed the wrong mix of files and want to re-pick what to add, or split one commit into several
`git reset --hard HEAD~1`	Remove last commit **and delete its changes**. **Use when:** you want to throw away a failed experiment completely. Dangerous
`git revert <sha>`	New commit that does the opposite of an old one; history kept. **Use when:** the bad commit is already pushed / on a shared or protected branch
`git reflog`	History of where HEAD has been. **Use when:** you lost commits after a reset --hard; then git reset --hard <sha> to recover

HEAD~1 = one commit back, HEAD~3 = three back; a SHA also works. Summary: **--soft** keeps changes staged, **mixed** keeps them unstaged, **--hard** deletes them.

10. Stash (park unfinished work)

Stash saves uncommitted changes on a stack and makes your working dir clean, without making a commit. **Use when:** you must switch branches for an urgent fix, or pull latest code, but your current work isn't ready to commit.

Command	What it does
`git stash / git stash push -m "msg"`	Save tracked uncommitted changes (with a label)
`git stash -u`	Also stash untracked (new) files
`git stash list`	Show saved stashes: stash@{0} is the newest
`git stash pop`	Re-apply newest stash and remove it from the list
`git stash apply stash@{1}`	Re-apply a stash but keep it in the list
`git stash show -p stash@{0}`	See what a stash contains
`git stash drop stash@{0} / git stash clear`	Delete one stash / all stashes
`git stash branch <name>`	Create a new branch from the stash and apply it there

11. GitHub & Open Source

- **Open source**: source code is public (Linux, Android, Mozilla, Apache, LibreOffice). **Closed/proprietary**: not public.
- **Star** = bookmark/popularity. **Watch** = get notified of changes. **Fork** = your own copy of someone's repo on your account.
- **Issues**: bugs, improvements or feature requests; labels classify them. Comment first to get the maintainer's go-ahead before working.
- **Pull Request (PR)**: request to merge your branch into the main repo; GitHub shows conflicts automatically.

- Other tabs: Discussions (forum), Actions (CI/CD automation), Projects (planning board), Insights (stats), Releases, Contributors.
- **Contribution flow:** Fork → `git clone` your fork → new branch → commit → push to your fork → open PR to the original repo → maintainer reviews and merges.
- **Protected main:** no direct push to main; changes need a PR, approvals and passing CI.
- **Review etiquette:** small focused PRs with clear descriptions; review the code, not the person; explain why; mark minor points as *nit*; respond quickly.

12. Quick Interview Q&A

Question	Answer
`Git vs GitHub`	Git = version control tool. GitHub = hosting platform for Git repos
`fetch vs pull`	fetch only downloads; pull = fetch + merge
`merge vs rebase`	merge keeps history and adds a merge commit; rebase replays your commits on top for a linear history. Don't rebase shared branches
`reset vs revert`	reset moves the branch back (rewrites history); revert adds an undo commit (safe after push)
`fork vs clone`	fork = copy on GitHub (server side); clone = copy to your PC
`What is HEAD?`	Pointer to the current branch's latest commit
`What is a SHA?`	40-character hash that uniquely identifies a commit
`Staging area?`	Lets you choose exactly which changes go into the next commit
`Fast-forward?`	Merge where the target branch has no new commits; pointer just moves ahead